Integración por Monte Carlo en memoria compartida con OpenMP

Sebastián Villamizar Saavedra
Código estudiantil 2230078 (semilla base del generador)

Introducción a la Programación Paralela
Universidad Industrial de Santander · 2026-2

Parte I. Fundamento del método

I.1. Error del estimador, 1/√N y costo de ganar un dígito

Se usa la variante de acierto y fallo: se generan puntos (x, y) uniformes en el cuadrado [0,1]×[0,1] y se cuenta la fracción que cae dentro del cuarto de círculo $x^2+y^2 \le 1$ (Figura 1).

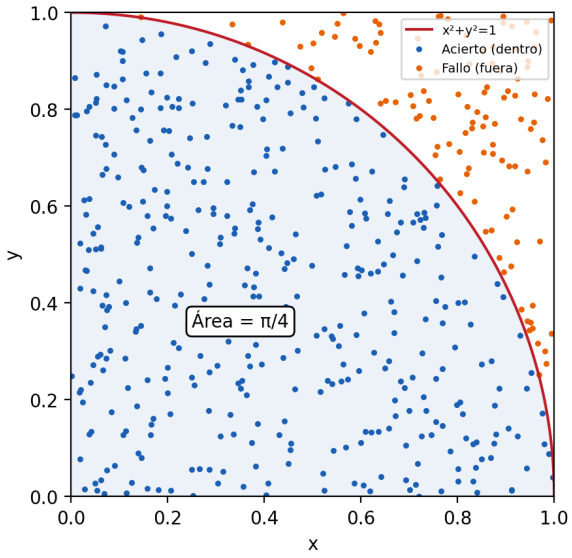


Figura 1. Variante de acierto y fallo (500 puntos generados con la semilla 2230078).

El cuadrado tiene área 1 y el cuarto de círculo π/4. Como los puntos son uniformes, la proporción de puntos dentro tiende a la proporción de área: π/4 ≈ M/N, de donde π ≈ 4·M/N, con M los aciertos y N los puntos totales.

Para saber cuánto se equivoca esta estimación se modela cada punto como un ensayo de Bernoulli independiente: X_i = 1 si el punto cae dentro del círculo y 0 en caso contrario. Llamo p = π/4 a la probabilidad teórica de acierto (no es lo mismo que $\hat{p}$ = M/N, la proporción observada en una corrida). Cada X_i tiene $E[X_i] = p$ y $Var(X_i) = p(1 - p)$.

La proporción observada es el promedio $\hat{p} = (1/N)\cdot\Sigma X_i$. Como los puntos son independientes, la varianza de la suma es la suma de las varianzas:

$$Var(\hat{p}) = (1/N^2) \cdot N \cdot p(1-p) = p(1-p)/N$$

Como $\hat{\pi} = 4\hat{p}$, $Var(\hat{\pi}) = 16\cdot p(1 - p)/N$ y el error típico (desviación estándar) es:

$$Error(N) = 4\cdot\sqrt{(p(1-p))/N} \approx 1{,}642/\sqrt{N} \quad \propto \quad 1/\sqrt{N}$$

Con sus palabras: cada punto nuevo aporta información, pero cada vez aporta menos. Las varianzas de muestras independientes se suman (crecen como N), mientras que el promedio divide entre N; al sacar la raíz para obtener la desviación, queda 1/√N. Los errores positivos y negativos de cada punto se cancelan parcialmente, y esa cancelación solo mejora con la raíz del número de muestras, no con N.

Muestras adicionales para reducir el error a la mitad. Se exige 1/√N*nuevo = 1/(2√N), es decir* √*N*nuevo = 2√N y N_nuevo = 4N. Las muestras adicionales son ΔN = 4N − N = **3N**.

Ganar un dígito decimal equivale a dividir el error entre 10: √N*nuevo = 10√N, o sea N*nuevo = **100N** (99N adicionales). El costo crece con el cuadrado de la precisión: por cada dígito decimal hay que

multiplicar el cómputo por 100. Por eso Monte Carlo sirve para aproximaciones rápidas de baja o mediana precisión, pero se vuelve muy costoso cuando se exige alta precisión de forma secuencial.

I.2. Por qué es un problema embarazosamente paralelo

Porque cada muestra es completamente independiente de las demás: generar un punto (x, y) y evaluar $x^2+y^2 \leq 1$ no necesita ningún resultado de otras muestras, ni comunicación ni sincronización durante el muestreo. Lo único que se acumula a lo largo del cálculo es el número de aciertos, y esa suma es asociativa y conmutativa, así que el trabajo se puede dividir en cualquier número de hilos casi sin coordinación.

Al repartir el muestreo con OpenMP hay que definir el ámbito de cada dato para evitar condiciones de carrera:

- **Privados por hilo:** las coordenadas temporales x, y; el índice de iteración; y sobre todo el estado interno del generador pseudoaleatorio (con rand_r(&estado) cada hilo avanza su propia copia). Si el estado fuera compartido, los hilos se pisarían y se perdería la independencia.
- **Compartidos (solo lectura):** el número total de muestras N, la semilla base y el número de bloques.
- **Acumulador de aciertos:** se gestiona con reduction(+:dentro). Cada hilo cuenta en una copia local y OpenMP las suma una sola vez al final, sin corromper el resultado.

I.3. El estimador paralelo es el mismo que el secuencial

En la versión secuencial se procesan N puntos y a cada punto m se le asigna $c_m = 1$ si $x_m^2 + y_m^2 \leq 1$ y 0 si no. Con C*sec = Σ c_m, la estimación es I*sec = 4·C_sec/N.

En la versión paralela las N muestras se reparten entre p hilos: el hilo k procesa un bloque de N*k muestras y obtiene C*k = Σ c*{k,i}. Los bloques son disjuntos (ninguna muestra la procesan dos hilos) y su unión es el conjunto completo (ninguna queda sin procesar), de modo que ΣN*k = N y cada par (k, i) corresponde exactamente a una muestra m. La reducción calcula C*par = ΣC*k.

Como la suma de enteros es asociativa y conmutativa, reagrupar los términos en sumas parciales no cambia el total: C*par = C*sec, y por tanto I*par = 4·C*par/N = I_sec. Esto vale cuando ambos programas evalúan los mismos puntos; si cada hilo usara un flujo aleatorio distinto, los números no coincidirían dígito a dígito, pero los estimadores seguirían siendo equivalentes en distribución (misma esperanza, misma varianza).

Precisión estadística: con el mismo N total cabe esperar la misma precisión que con un solo hilo. El error $1/\sqrt{N}$ depende solo del número total de muestras independientes, no de cuántos hilos las procesan. Más hilos reducen el tiempo, no el error. Esto supone flujos independientes y sin solapamiento; si estuvieran correlacionados, las muestras efectivas serían menos de N y el error real sería mayor (se demuestra en II.3).

Parte II. Implementación en OpenMP

Todo se ejecutó en mi equipo AMD Ryzen 5 5500U (6 núcleos físicos, 12 hilos lógicos), con Windows y MSYS2 UCRT64 (gcc 16.2.0), con N = 10^8 puntos y la semilla 2230078 (mi código estudiantil).

Decisión de portabilidad. El enunciado usa rand*r de glibc. En MSYS2/MinGW rand*r no existe (o usa otro algoritmo), así que implementé en el propio programa la función mi*rand*r, que es exactamente el algoritmo de glibc (tres pasos del LCG $s \leftarrow 1103515245 \cdot s + 12345 \bmod 2^{32}$ por llamada). Verifiqué que produce lo mismo que el rand*r del sistema: en Linux, el programa original del enunciado con rand*r dio 78 538 650 aciertos con la semilla 2230078, igual que mi versión; y con la semilla del trabajo grupal (20261018) reproduce los 78 540 833 aciertos que habíamos obtenido.

II.1 Versión secuencial con N = 10^8

Compilé y ejecuté el programa del enunciado (con la semilla 2230078): **$\pi \approx 3{,}141546$** con 78 538 650 aciertos. El valor real es 3,141593, así que el error absoluto es $4{,}67 \times 10^{-5}$.

¿De dónde sale el orden de 10^{-4}? El resultado depende del azar y el error baja como $1/\sqrt{N}$. Con N = 10^8, $\sqrt{N} = 10^4$, así que el error esperado ronda $1/10^4 = 10^{-4}$ (la fórmula exacta de I.1 da $\sigma = 4 \cdot \sqrt{p(1-p)/N} = 1{,}64 \times 10^{-4}$). Mi error ($4{,}67 \times 10^{-5}$) es 0,28σ, dentro de lo normal: el resultado es correcto.

Tiempo: medí con omp*get*wtime() en 7 corridas y tomé la mediana: **T(1) = 1,0570 s**. Seis de las siete corridas quedaron entre 1,044 y 1,070 s y una tardó solo 0,837 s; la mediana ignora ese valor extremo. Esta medición se hizo con el plan de energía normal; en la Parte III repetí con Máximo rendimiento y T(1) bajó a 0,6270 s (ver allí). Por eso la Parte III usa su propio T(1).

```
sebas@SEBASTIAN UCRT64 /c/Users/sebas/OneDrive/Documentos/codigo_Villamizar_2230078/codigo
$ bash scripts/compilar.sh
gcc.exe (Rev3, Built by MSYS2 project) 16.2.0
$ gcc -O2 -fopenmp montecarlo_ref.c -o montecarlo_ref -lm
$ gcc -O2 -Wall -Wextra -fopenmp montecarlo.c -o montecarlo -lm
$ gcc -O2 -fopenmp -DPLAN=dynamic montecarlo.c -o montecarlo_dynamic -lm
$ gcc -O2 -fopenmp pruebas/estado_compartido.c -o estado_compartido -lm
$ gcc -O2 -fopenmp pruebas/misma_semilla.c -o misma_semilla -lm
$ gcc -O2 -fopenmp pruebas/sin_reduccion.c -o sin_reduccion -lm
$ gcc -O2 -fopenmp pruebas/semillas_hilos.c -o semillas_hilos -lm
(todo compilo sin advertencias)
```

Figura II-1. Todo compila sin errores ni advertencias.

```
sebas@SEBASTIAN UCRT64 /c/Users/sebas/OneDrive/Documentos/codigo_Villamizar_2230078/codigo
$ bash scripts/ii1_secuencial.sh
=== Version secuencial del enunciado, semilla 2230078 (7 corridas) ===
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=1.0700 s
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=1.0610 s
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=1.0490 s
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=1.0580 s
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=1.0570 s
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=1.0440 s
secuencial N=100000000 dentro=78538650 pi=3.141546 |error|=4.67e-05 T=0.8370 s
T(1) = mediana de las 7 corridas = 1.0570 s

=== Error tipico esperado ===
q = pi/4 = 0.785398  sigma = 4*sqrt(q(1-q)/N) = 1.64e-04   1/sqrt(N) = 1e-04
```

Figura II-2. II.1: π = 3,141546 y T(1) = 1,0570 s (mediana de 7 corridas).

II.2 Versión paralela con OpenMP

No cambié el programa original, solo repartí el trabajo. Dividí los 10^8 puntos en 1000 bloques de 100 000 y OpenMP reparte los bloques entre los hilos (código completo en el Anexo II-A):

```
#pragma omp parallel for reduction(+:dentro) schedule(static)
for (long long b = 0; b < nbloques; b++) {
    unsigned int estado = saltar(semilla, b * BLOQUE * 6);
    /* aqui va el mismo ciclo del programa original */
}
```

- **Reducción:** reduction(+:dentro) da a cada hilo su propio contador, que se suman al final.
- **Elegir hilos:** como argumento (./montecarlo 100000000 4, que llama omp*set*num*threads) o con OMP*NUM_THREADS.
- **Flujos independientes y sin solapamiento:** los números salen de una única lista fija que empieza en la semilla 2230078. La función saltar() avanza el LCG k pasos en O(log k) operaciones, y lleva cada bloque directo a su sitio en la lista. Con p = 4 cada hilo empieza N/p = 25 millones de puntos después del anterior (estados iniciales 2230078, 693688254, 2781521982 y 1735882942; coinciden con avanzar el generador paso a paso, Figura II-3), que es el espaciado que pide el enunciado.
- **Reproducibilidad:** cada bloque usa siempre sus mismos números, sin importar qué hilo lo ejecute. Con p = 1, 2, 4 y 8 (y también 3 y 7) obtuve siempre **78 538 650 aciertos**, igual que el programa secuencial. El tiempo bajó de 1,0720 s con 1 hilo a 0,5730 s (2 hilos), 0,3120 s (4 hilos) y 0,1580 s (8 hilos).

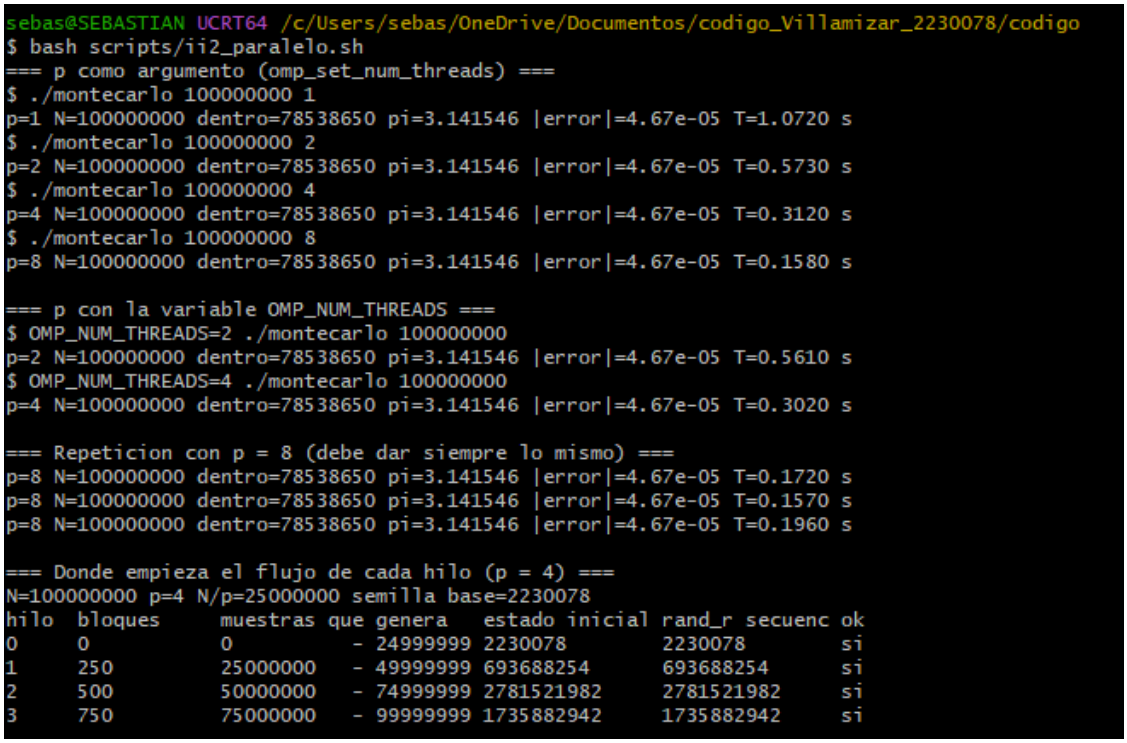


Figura II-3. Mismo resultado con 1, 2, 4 y 8 hilos; abajo, dónde empieza el flujo de cada hilo con p = 4.

II.3 Por qué no es correcto compartir el estado global de rand()

rand() guarda su posición en un único lugar compartido por todos los hilos. Eso causa dos problemas distintos, que probé con programas hechos a propósito (carpeta pruebas/):

1. Condición de carrera sobre el estado interno. Avanzar el generador es leer, calcular y guardar el estado; si dos hilos lo hacen a la vez sobre la misma variable, es una carrera de datos y el comportamiento queda indefinido: pueden tomar el mismo número o saltarse otros, y el resultado ya no es el de ningún flujo aleatorio válido.

- **Lo que observé:** con 8 hilos las tres ejecuciones dieron 78 525 936 aciertos ($\pi = 3{,}141037$), no el valor correcto 78 538 650 que da 1 hilo. Es decir, el resultado es incorrecto, y el programa no tardó más (0,16 a 0,21 s). Este resultado coincide exactamente con el del problema 2 ($8 \times 9\,815\,742$). Mi explicación, que no verifiqué mirando el ensamblador, es que con -O2 el compilador mantuvo el estado en un registro de cada hilo, de modo que cada hilo recorrió el mismo flujo desde la misma semilla. Es una consecuencia típica de la carrera de datos: no hay garantía de qué se observa, y que sea repetible en mi compilación no la hace correcta. En otra máquina o con otro compilador el síntoma puede ser distinto (resultados diferentes en cada corrida).
- En π y en el error: la estimación sale sesgada y, como se repiten números, hay menos puntos independientes de los que creemos; el error real ($5{,}55\times10^{-4}$) es mayor que el reportado ($\sigma \approx 1{,}64\times10^{-4}$).

2. La misma semilla en todos los hilos. Si cada hilo tiene su propio generador pero todos empiezan en 2230078, todos lanzan exactamente los mismos puntos.

- En π: con 8 hilos obtuve **3,141037** (78 525 936 aciertos), idéntico a usar solo N/8 = 12,5 millones de puntos con un hilo (9 815 742 aciertos × 8). Copiar los mismos puntos 8 veces no ayuda en nada.
- En el error: parece que tengo la precisión de 10^8 puntos ($\pm1{,}64\times10^{-4}$), pero el error real fue **$5{,}55\times10^{-4}$**, unas 3,4 veces peor. Las muestras efectivas son N/8, no N.

Mi versión evita ambos problemas: cada hilo tiene su propio estado privado y cada bloque empieza en un lugar distinto de la lista.

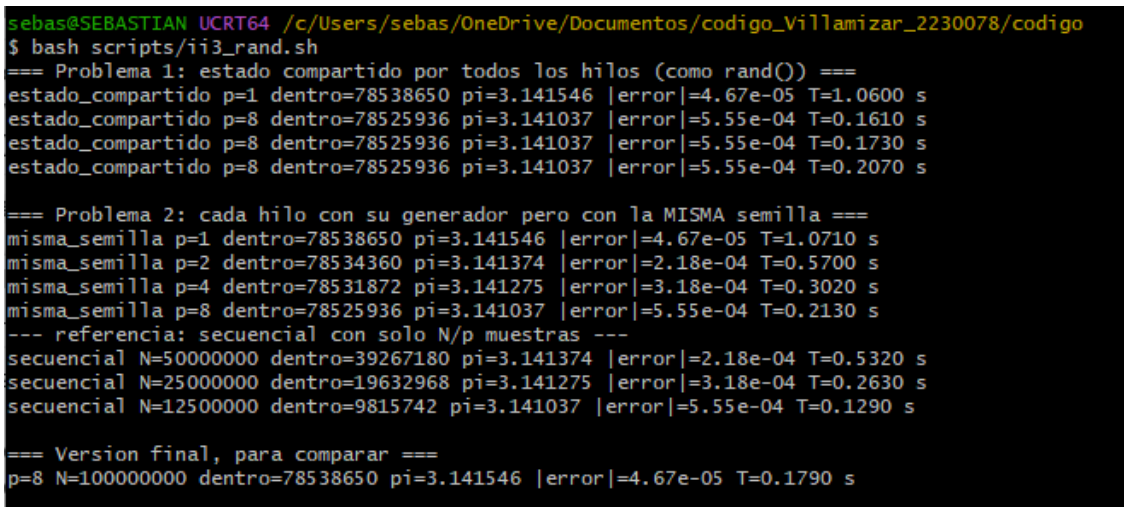


Figura II-4. Con estado compartido el resultado cambia y es más lento; con la misma semilla, 8 hilos dan lo mismo que N/8 puntos.

II.4 Atributos de compartición de datos

Una variable compartida es la misma para todos los hilos; una privada es una copia propia de cada hilo. Para no adivinar, usé default(none) en una copia del código (pruebas/default_none.c): el compilador se queja de cada variable que no clasifiqué, y nombró N, nbloques y semilla (Figura II-5).

Variable	Tipo	Por qué
N, nbloques, semilla	compartidas	se crean antes del ciclo y solo se leen
b	privada	es el contador del ciclo paralelo
estado, fin, i, x, y	privadas	se declaran dentro del ciclo
dentro	privada gracias a reduction	es el contador de aciertos

Que N, nbloques y semilla sean compartidas no causa problemas porque nadie las modifica.

Cláusula usada: reduction(+:dentro). Cada hilo cuenta sus aciertos en una copia propia inicializada en 0, y al final OpenMP suma todas las copias.

Sin esa cláusula, dentro sería compartida por defecto y todos los hilos incrementarían el mismo contador. Un dentro++ son tres pasos (leer, sumar, guardar); si dos hilos lo hacen a la vez, uno borra la suma del otro. Es una **condición de carrera (data race)** y se pierden aciertos. Con 1 hilo da bien (78 538 650), por eso el error no se nota si solo se prueba sin paralelismo. Con 8 hilos obtuve π = 0,392751; 0,392785 y 0,392687 (9 818 771; 9 819 628 y 9 817 165 aciertos): distinto en cada ejecución y muy lejos de 3,1416. Se perdió cerca del 87,5 % de los aciertos: el valor final es aproximadamente el de un solo hilo (78 538 650 / 8 ≈ 9,82 millones), porque las actualizaciones de los demás se pisan.

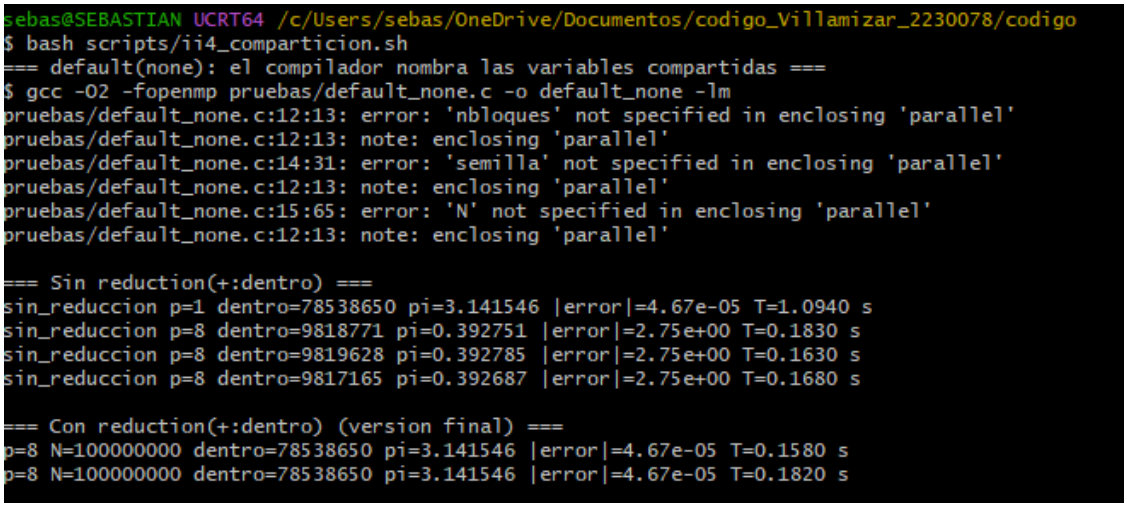


Figura II-5. El compilador nombra las variables compartidas; sin reduction π sale mal; con reduction sale bien.

Anexo II-A. Programa paralelo completo (montecarlo.c)

```
/* montecarlo.c -- version paralela con OpenMP (version final).
   Autor: Sebastian Villamizar Saavedra - codigo 2230078 (semilla base).
   Uso: ./montecarlo <numero de muestras> [hilos] [semilla]
        (el numero de hilos tambien puede fijarse con OMP_NUM_THREADS)
   Compilar: gcc -O2 -fopenmp montecarlo.c -o montecarlo -lm

   Idea: la secuencia de rand_r con la semilla base se parte en bloques
   consecutivos de BLOQUE muestras. El estado inicial de cada bloque se
   calcula "saltando" hacia adelante en el generador, asi que cada bloque
   consume siempre los mismos numeros, sin importar que hilo lo ejecute.
   Resultado: flujos disjuntos (sin solapamiento) y la misma estimacion
   para cualquier p, identica a la de la version secuencial.

   Nota de portabilidad: rand_r se implementa aqui mismo con el algoritmo
   de glibc (mi_rand_r). Asi el programa da los mismos numeros en Linux,
   WSL o MSYS2/MinGW, donde rand_r no existe o es distinta. */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

#define SEMILLA_BASE 2230078u  /* codigo estudiantil: Sebastian Villamizar */
#define BLOQUE 100000LL        /* muestras por bloque */
#define RAND_MAX_GLIBC 2147483647.0

#ifndef PLAN
#define PLAN static            /* -DPLAN=dynamic para otra planificacion */
#endif

/* rand_r de glibc: 3 pasos del LCG s <- 1103515245*s + 12345 (mod 2^32)
   por llamada; el resultado combina 31 bits de los tres estados. */
#define LCG_A 1103515245u
#define LCG_C 12345u
#define PASOS_POR_MUESTRA 6ULL   /* 2 llamadas (x, y) x 3 pasos */

static int mi_rand_r(unsigned int *seed) {
    unsigned int next = *seed;
    int result;
    next = next * LCG_A + LCG_C;
    result = (int)((next / 65536u) % 2048u);
    next = next * LCG_A + LCG_C;
    result <<= 10;
    result ^= (int)((next / 65536u) % 1024u);
    next = next * LCG_A + LCG_C;
    result <<= 10;
    result ^= (int)((next / 65536u) % 1024u);
    *seed = next;
    return result;
}

/* Avanza el estado s en k pasos del LCG con O(log k) operaciones. */
static unsigned int saltar(unsigned int s, unsigned long long k) {
```

```
    unsigned int a = LCG_A, c = LCG_C;  /* transformacion de 2^j pasos */
    unsigned int A = 1u, C = 0u;        /* transformacion acumulada */
    while (k) {
        if (k & 1ULL) {
            A = a * A;
            C = a * C + c;
        }
        c = a * c + c;
        a = a * a;
        k >>= 1;
    }
    return A * s + C;
}

int main(int argc, char *argv[]) {
    long long N = (argc > 1) ? atoll(argv[1]) : 100000000L;
    if (argc > 2) omp_set_num_threads(atoi(argv[2]));
    unsigned int semilla = (argc > 3) ? (unsigned int)strtoul(argv[3], NULL, 10)
                                      : SEMILLA_BASE;

    /* Verifica que el salto coincide con lo que hace mi_rand_r. */
    unsigned int prueba = semilla;
    mi_rand_r(&prueba);
    mi_rand_r(&prueba);
    if (prueba != saltar(semilla, PASOS_POR_MUESTRA)) {
        fprintf(stderr, "error: el salto del LCG no coincide con mi_rand_r\n");
        return 1;
    }

    /* Periodo del estado: 2^32. Con mas de 2^32/6 muestras la secuencia se
       repetiria y los bloques dejarian de ser disjuntos. */
    if ((unsigned long long)N * PASOS_POR_MUESTRA > 4294967296ULL)
        fprintf(stderr, "aviso: N supera el periodo del generador\n");

    long long nbloques = (N + BLOQUE - 1) / BLOQUE;
    long long dentro = 0;

    double t0 = omp_get_wtime();
    #pragma omp parallel for reduction(+:dentro) schedule(PLAN)
    for (long long b = 0; b < nbloques; b++) {
        unsigned int estado =
            saltar(semilla, (unsigned long long)b * BLOQUE * PASOS_POR_MUESTRA);
        long long fin = (b + 1) * BLOQUE < N ? (b + 1) * BLOQUE : N;
        for (long long i = b * BLOQUE; i < fin; i++) {
            double x = mi_rand_r(&estado) / RAND_MAX_GLIBC;
            double y = mi_rand_r(&estado) / RAND_MAX_GLIBC;
            if (x * x + y * y <= 1.0)
                dentro++;
        }
    }
    double t = omp_get_wtime() - t0;

    double pi = 4.0 * dentro / (double)N;
    printf("p=%d N=%lld dentro=%lld pi=%.6f |error|=%.2e T=%.4f s\n",
           omp_get_max_threads(), N, dentro, pi, fabs(pi - M_PI), t);
    return 0;
}
```

Parte III. Evaluación de desempeño: speedup y ley de Amdahl

Equipo. Las mediciones se hicieron en un portátil con un AMD Ryzen 5 5500U: 6 núcleos físicos con 2 hilos cada uno (12 hilos lógicos), 2,1 GHz de frecuencia base reportada por Windows y 8 MB de caché L3 (salida de scripts/cpu.sh en la Figura III-3). Es un procesador homogéneo (no tiene núcleos de rendimiento y de eficiencia mezclados). Se usó Windows con el plan de energía en Máximo rendimiento, MSYS2 UCRT64 y gcc 16.2.0.

Procedimiento. Usé el programa final de la Parte II (1000 bloques de 100 000 muestras, schedule(static)), compilado con gcc -O2 -fopenmp montecarlo.c -o montecarlo -lm, con N = 10^8 y la semilla 2230078. Con scripts/iii*medir.sh ejecuté una corrida de calentamiento y luego cinco corridas para cada p ∈ {1, 2, 4, 8}. El tiempo lo mide el programa con omp*get_wtime() alrededor del ciclo paralelo. En las 20 corridas el programa dio los mismos 78 538 650 aciertos (π = 3,141546), así que las diferencias de tiempo vienen solo de la ejecución.

Una primera tanda que descarté. Mi primera tanda, con el plan de energía normal, dio T(1) = 1,0600 s y speedups mayores que p (S(4) = 4,84 y S(8) = 8,41), con lo que Amdahl daba f = 1,058, una fracción imposible. Eso indicaba que T(1) estaba inflado: en II.1 ya había visto una corrida secuencial de 0,837 s entre otras de 1,05 s, y p = 2 se separaba en dos grupos (0,39 y 0,56 s). Repetí todo con el plan en Máximo rendimiento: T(1) bajó de 1,060 s a 0,6270 s, mientras que p = 8 casi no cambió (0,1260 s → 0,1240 s). Es decir, solo las corridas con pocos hilos estaban frenadas por la gestión de energía, y por eso el speedup salía inflado. La tabla siguiente es la tanda válida. Las capturas de la Parte II se tomaron con el plan normal, así que sus tiempos absolutos son mayores; para S(p) solo uso esta tanda, medida toda en las mismas condiciones.

¿Por qué la mediana y no el promedio? El ruido de un portátil no es simétrico: una tarea en segundo plano o un cambio de frecuencia pueden hacer que una corrida tarde bastante más y arrastran el promedio. La mediana es el valor central de los tiempos ordenados; con cinco corridas puede haber hasta dos atípicas sin que cambie. En mis datos se ve con p = 4: una corrida tardó 0,2090 s por encima de las otras cuatro (0,1940 a 0,2030 s); el promedio sube a 0,2004 s y la mediana se queda en 0,1980 s. Con p = 2 pasa algo parecido (0,3390 s frente a 0,3190–0,3240 s; promedio 0,3254 s, mediana 0,3240 s). Aquí la diferencia es pequeña porque las mediciones fueron estables (la mayor dispersión fue 0,023 s, con p = 1; con p = 8 todo cayó entre 0,1240 y 0,1250 s), pero con una sola corrida interrumpida por el sistema operativo el promedio se movería mucho más y la mediana no.

Tabla III-1. Tiempos medidos con N = 10^8 en el AMD Ryzen 5 5500U, plan Máximo rendimiento (cinco corridas por valor de p). S(p) y E(p) se calculan con T(1) = 0,6270 s.

p (hilos)	Tiempos medidos [s]	Mediana de T(p) [s]	S(p) = T(1)/T(p)	E(p) = S(p)/p
1	0,6200 · 0,6210 · 0,6270 · 0,6350 · 0,6430	0,6270	1,000	1,000
2	0,3190 · 0,3210 · 0,3240 · 0,3240 · 0,3390	0,3240	1,935	0,968
4	0,1940 · 0,1980 · 0,1980 · 0,2030 · 0,2090	0,1980	3,167	0,792
8	0,1240 · 0,1240 · 0,1240 · 0,1250 · 0,1250	0,1240	5,056	0,632

El programa escala bien pero no de forma ideal: con 2 hilos la eficiencia es 0,97, con 4 baja a 0,79 y con 8 a 0,63. Con 8 hilos el cálculo pasa de 0,6270 s a 0,1240 s, 5,1 veces más rápido.

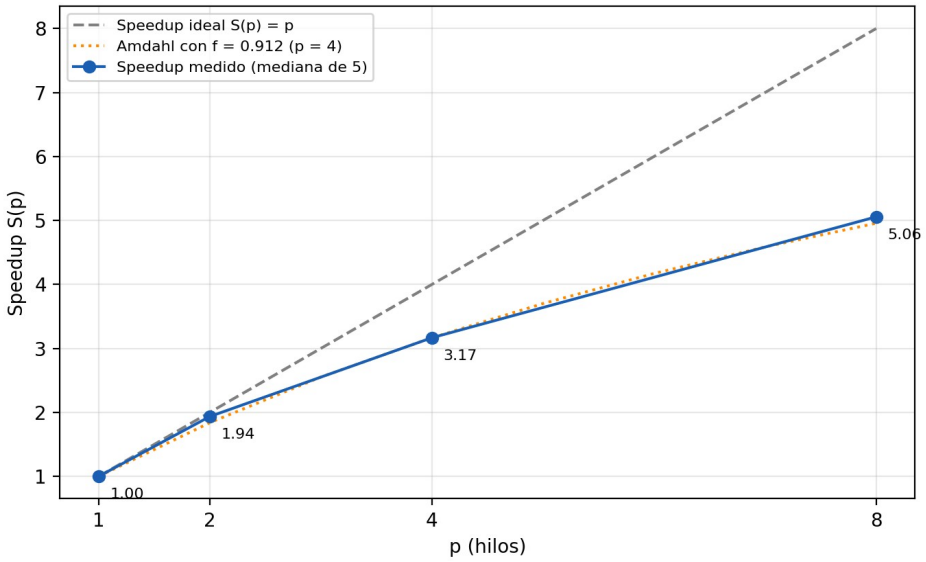


Figura III-1. Speedup medido frente al ideal S(p) = p y a la ley de Amdahl con la f estimada en III.2.

III.1 Ley de Amdahl: tres casos

(a) f = 0,98, p = 8. S = 1 / (0,02 + 0,98/8) = 1 / (0,02 + 0,1225) = 1 / 0,1425 ≈ **7,02**, con eficiencia 7,02/8 ≈ 0,88. Aunque solo el 2 % del trabajo sea secuencial, ya se pierde el 12 % de la capacidad de los 8 hilos: en el denominador ese 2 % pesa 0,02 y el 98 % repartido pesa 0,1225, así que la parte secuencial ya es casi la sexta parte del tiempo total.

(b) f = 0,90, p = 8. S = 1 / (0,10 + 0,90/8) = 1 / 0,2125 ≈ **4,71**, con eficiencia ≈ 0,59. Pasar de 2 % a 10 % de código secuencial baja el speedup de 7,02 a 4,71: con 8 hilos la parte secuencial (0,10) ya cuesta casi lo mismo que toda la parte paralela (0,1125).

(c) f = 0,98 y p → ∞. El término f/p tiende a cero y S → 1/(1 − f) = 1/0,02 = **50**. Por más núcleos que se agreguen, el programa nunca será más de 50 veces más rápido.

Lo que enseña (c): el techo lo pone la fracción secuencial y no el número de procesadores. Esa parte la ejecuta un solo hilo, así que el tiempo nunca baja de (1 − f)·T(1). Reducir lo secuencial vale más que agregar núcleos: bajarlo de 2 % a 1 % sube el techo de 50 a 100.

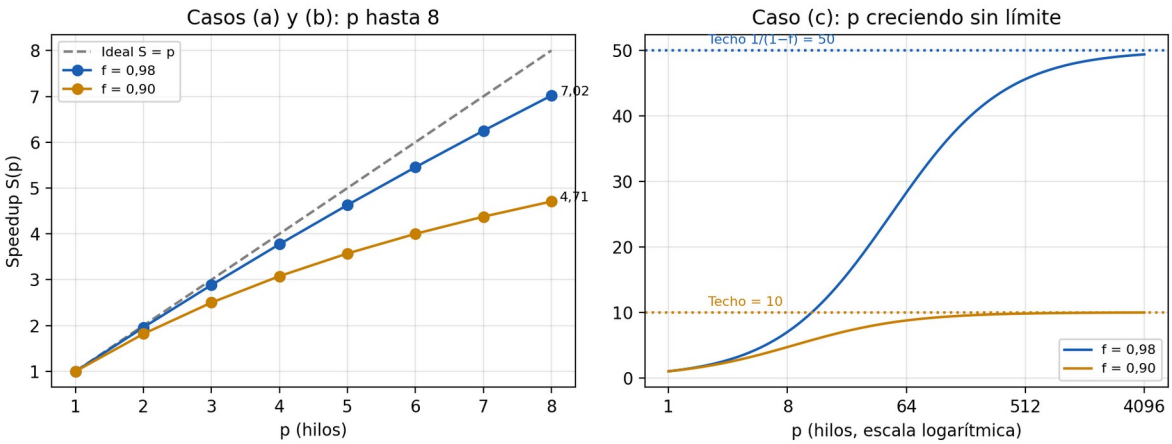


Figura III-2. Ley de Amdahl para f = 0,98 y f = 0,90. A la izquierda, casos (a) y (b); a la derecha, caso (c): S se acerca a 1/(1 − f) y nunca lo supera.

III.2 Estimación de la fracción f

Invierto la ley de Amdahl y agrupo los términos con f:

$$1/S = (1 - f) + f/p = 1 - f\cdot(1 - 1/p) \quad \Longrightarrow \quad f = (1 - 1/S) \,/\, (1 - 1/p)$$

Cálculo con p = 4 y las medianas de la Tabla III-1: S(4) = T(1)/T(4) = 0,6270 / 0,1980 = 3,167; 1/S = 0,3158; numerador 1 − 0,3158 = 0,6842; denominador 1 − 1/4 = 0,75; **f = 0,6842 / 0,75 = 0,912**, es decir, una fracción “secuencial” efectiva de 1 − f = 0,088 (8,8 %).

¿Es razonable? Fuera del ciclo el código solo lee argumentos, verifica que saltar coincide con mi*rand*r, calcula el número de bloques e imprime: microsegundos frente a 0,63 s de cómputo, menos del 0,01 %. Así que el 8,8 % no corresponde a instrucciones secuenciales: es una fracción efectiva que mezcla costos que Amdahl no distingue. Los candidatos principales son la frecuencia del procesador (un núcleo solo puede rendir más que varios activos a la vez) y el hiperprocesamiento (con 8 hilos y 6 núcleos físicos, algunos hilos comparten núcleo).

Karp–Flatt. e = (1/S − 1/p)/(1 − 1/p) estima la fracción secuencial experimental: incluye el código secuencial y todo lo que impide escalar (creación y sincronización de hilos, desbalance de carga, pérdida de frecuencia). Con p = 4 coincide con 1 − f porque es el mismo despeje. Lo útil es calcularla para varios p:

p	S(p)	e (Karp–Flatt)	f = 1 − e
2	1,935	0,033	0,967
4	3,167	0,088	0,912
8	5,056	0,083	0,917

Si la pérdida viniera de una parte secuencial fija del código, e sería más o menos constante. Aquí e es pequeña con 2 hilos (0,033), casi se triplica con 4 (0,088) y se queda en ≈ 0,083 con 8. El salto entre 2 y 4 hilos indica que a partir de cierto número de hilos aparece un costo que con 2 no existe, y que después se estabiliza. No puedo decir cuál es la causa solo con estos datos: encaja con la bajada de frecuencia cuando hay más núcleos activos y con que el sistema operativo coloque dos de mis hilos en el mismo

núcleo físico, pero no medí ni la frecuencia ni en qué núcleo corrió cada hilo. Por eso el f de Amdahl depende del p con el que se calcule; sigo con el que pide el enunciado, f = 0,912.

III.3 Speedup medido vs. ideal vs. Amdahl

p	Ideal S = p	Amdahl (f = 0,912)	Medido	Medido / Amdahl
2	2	1,839	1,935	1,05
4	4	3,167	3,167	1,00
8	8	4,957	5,056	1,02

El speedup medido siempre queda por debajo del ideal y la distancia crece con p: con 8 hilos se obtiene 5,06 en lugar de 8. Frente a Amdahl con f = 0,912 la diferencia es pequeña: con 2 hilos el programa escala un 5 % mejor que lo predicho y con 8 hilos un 2 % mejor. Que Amdahl con un f tan alto acierte bien indica que, para $p \in \{2, 4, 8\}$, la pérdida se comporta como una fracción casi fija. Factores, evaluados para este programa:

- **Creación y destrucción de hilos.** Hay una sola región paralela; crear el equipo cuesta del orden de microsegundos frente a más de 0,1 s de cómputo. Calcular los saltos del generador tampoco pesa: son 1000 saltos de unas cien multiplicaciones cada uno, frente a 2×10^8 llamadas a mi*rand*r. No explica las diferencias.
- **Costo de la reducción.** Cada hilo cuenta en su copia privada y OpenMP suma las p copias una sola vez: como máximo 8 sumas. Despreciable (a diferencia de II.4 sin la cláusula, donde la variable compartida hacía perder aciertos).
- **Ancho de banda de memoria y jerarquía de caché.** El ciclo no recorre arreglos: el estado, x, y y el contador caben en registros. El programa está limitado por cálculo y no por memoria, así que ni el ancho de banda ni la caché L3 compartida de 8 MB son un cuello de botella; tampoco hay false sharing, porque cada hilo escribe solo en su copia privada.
- **Frecuencia variable.** Mi primera tanda lo mostró con claridad: con el plan de energía normal T(1) fue 1,06 s y con Máximo rendimiento 0,627 s, con el mismo código. La frecuencia de la CPU puede cambiar con la carga, la temperatura y el plan de energía. Con un solo hilo activo, un núcleo puede alcanzar frecuencias altas; con varios activos el límite de potencia de un portátil obliga a bajarlas. Esto reduce S(p) aunque el programa sea casi totalmente paralelo.
- **Hiperprocesamiento.** El Ryzen tiene 6 núcleos físicos con 2 hilos cada uno. Con p = 8 algún núcleo ejecuta dos hilos míos, que compiten por las mismas unidades de cálculo; en un ciclo tan aritmético cada uno avanza más lento que en un núcleo propio. Como referencia, con reparto estático el tiempo lo fija el hilo más lento: para que S(8) = 5,06, los hilos que comparten núcleo tendrían que rendir a ≈ 63 % de su velocidad (5,06/8), lo que equivale a unos 1,26 hilos de trabajo por núcleo, una ganancia típica del hiperprocesamiento. Es una estimación compatible con los datos, no una medición.
- **Desbalance de carga.** Con schedule(static) cada hilo recibe la misma cantidad de bloques (500, 250 y 125 para p = 2, 4 y 8). Si un hilo cae en un núcleo compartido o es interrumpido, termina después y los demás lo esperan antes de la reducción. Una planificación dynamic podría compensarlo (el binario montecarlo_dynamic quedó compilado, pero no lo medí).

- **Sistema operativo.** Los hilos compiten con el resto de procesos de Windows; cualquier tarea de fondo retrasa a un hilo y los demás deben esperarlo. En la tanda válida casi no se notó (p = 8 varió solo entre 0,1240 y 0,1250 s).

III.4 Máquina de 64 núcleos

Con f = 0,912 de III.2: $S(64) = 1 / ((1 - f) + f/64) = 1 / (0{,}088 + 0{,}01425) \approx$ **9,8**. El tiempo pasaría de 0,627 s a unos 0,064 s, con una eficiencia de 9,8/64 ≈ 0,15, y el techo para cualquier número de núcleos sería $1/(1 - f) \approx 11{,}4$. Hay que leer este número con cuidado: mi f incluye efectos propios de mi portátil (frecuencia variable, hiperprocesamiento) que un servidor de 64 núcleos iguales y con frecuencia estable no tendría. Allí f sería mucho más cercano a 1: con f = 0,98 el speedup sería ≈ 28, y con el f del código (≈ 0,999) ≈ 60. También habría un límite propio del programa: con 1000 bloques y 64 hilos, unos hilos reciben 16 bloques y otros 15, todos esperan a los de 16 y la eficiencia no puede pasar de 15,625/16 ≈ 0,98. Con más núcleos convendría usar bloques más pequeños.

Aumentar p con N fijo no mejora la precisión. En las 20 corridas de la Tabla III-1 el programa dio exactamente los mismos 78 538 650 aciertos con p = 1, 2, 4 y 8: más hilos procesan los mismos puntos, solo que en menos tiempo. El error depende solo de N: $\sigma = 4 \cdot \sqrt{[(\pi/4)(1 - \pi/4)/N]} \approx 1{,}64/\sqrt{N}$. Con $N = 10^8$, $\sigma \approx 1{,}64 \times 10^{-4}$, y mi error real es $4{,}67 \times 10^{-5}$ sea cual sea p. Con 64 núcleos y el mismo N tendría exactamente el mismo π, solo que más rápido.

Qué mejora cada variable. Aumentar p reduce el tiempo; aumentar N reduce el error, pero lentamente (por I.1: mitad del error exige 4N muestras y un decimal exige 100N). La forma de aprovechar los 64 núcleos para ganar precisión es combinar ambas: usar 64 veces más muestras ($N = 6{,}4 \times 10^9$) en un tiempo parecido al actual. Eso divide el error entre $\sqrt{64} = 8$, hasta $\approx 2 \times 10^{-5}$: menos de un decimal adicional a cambio de 64 veces más hardware. Para ese N el programa necesitaría un cambio: el LCG de rand_r repite su secuencia tras 2^{32} pasos y cada muestra usa 6, así que solo es válido hasta $N \approx 7{,}2 \times 10^8$ (por eso imprime un aviso si N lo supera). Habría que usar un generador de 64 bits al que se le pueda aplicar la misma técnica de salto.

III.5 (Opcional) Amdahl pesimista y el argumento de Gustafson

Amdahl supone N fijo y pregunta cuánto baja el tiempo al agregar núcleos. Pero con más núcleos normalmente no se resuelve el mismo problema más rápido, sino uno más grande en el mismo tiempo (como en III.4, al pasar a $N = 6{,}4 \times 10^9$). Si s es la fracción secuencial medida en la ejecución paralela, el speedup escalado de Gustafson es:

$$S_G(p) = s + (1 - s) \cdot p = p - s \cdot (p - 1)$$

Al crecer N, la parte paralela (el ciclo de muestreo) crece en proporción y la secuencial (leer argumentos, verificar el generador, imprimir) sigue igual, así que s se hace cada vez más pequeña y deja de imponer un techo fijo. Con mi $s = 1 - f = 0{,}088$ y p = 64: $S_G = 64 - 0{,}088 \cdot 63 \approx$ **58,5**, frente a 9,8 de Amdahl. Como mi s mezcla efectos del hardware y no código secuencial real, ambos números son cotas para mi portátil y no una predicción para otra máquina. En Monte Carlo esta visión es la más natural, porque el motivo para usar más núcleos es procesar más muestras y reducir el error (Parte I); por eso Amdahl resulta pesimista aquí.

Anexo III. Identificación de la CPU

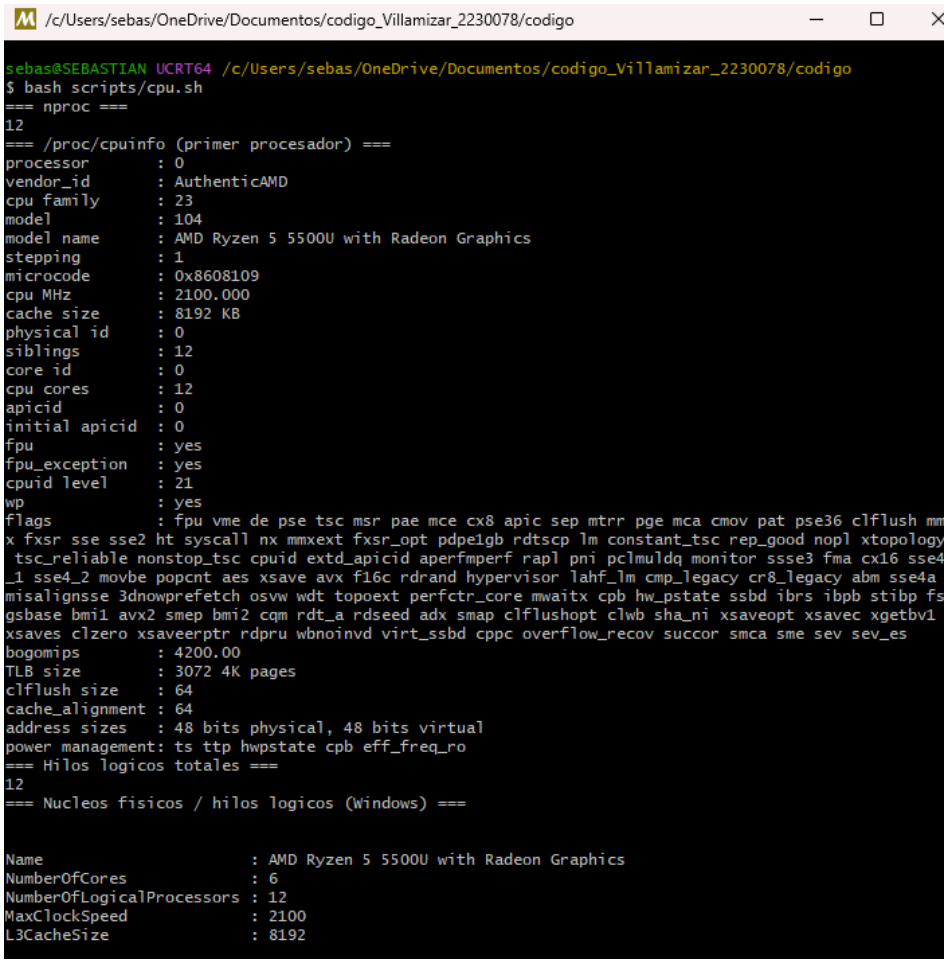


Figura III-3. Salida de scripts/cpu.sh: AMD Ryzen 5 5500U, 6 núcleos físicos, 12 hilos lógicos.

Bitácora de uso de IA

Incluye las consultas del trabajo grupal original (semilla 20261018) y las de esta versión individual (semilla 2230078).

#	Herramienta	Propósito	Qué aportó	Qué se aceptó / descartó y por qué
1	Claude (grupo)	Figura 1 de acierto y fallo	Código en Python para dibujar la variante acierto y fallo	Aceptado como apoyo visual; en esta versión regeneré la figura con la semilla 2230078.
2	ChatGPT (grupo)	Aclarar conceptos de Monte Carlo	Explicación del método de acierto y fallo	Aceptado para entender el problema; la redacción de I.1–I.3 es propia.
3	Gemini (grupo)	Lógica del ensayo de Bernoulli	Pasos matemáticos para llegar a $1/\sqrt{N}$	Aceptado; verifiqué la deducción de varianza a mano.
4	Perplexity (grupo)	Especificaciones de OpenMP para I.2	Citas de la especificación sobre ámbito de variables	Aceptado como referencia; revisé el enlace oficial.
5	Claude (grupo)	Comparar secuencial vs. paralelo	Ejemplos de que el estimador se mantiene	Aceptado.
6	Claude Code, Opus 5.5 (grupo, 26/10/2026)	Parte II del trabajo grupal	Programas de prueba, scripts y borrador de respuestas	Aceptado: una prueba por defecto. Descartado: T(1) medido con el portátil en batería.
7	Claude Sonnet 5.5 (individual, 27/10/2026)	Adaptar el trabajo a la entrega individual	montecarlo.c con semilla 2230078, mi*rand*r portable (algoritmo de glibc) para MSYS2, programas de prueba, scripts de medición y análisis, borrador de este documento	Aceptado: el código, que verifiqué compilando y comparando con rand_r (78 538 650 aciertos), y el diagnóstico de la primera tanda de tiempos en la Parte III. Descartado: los datos de tiempos y de la CPU del trabajo grupal, porque no eran de mi equipo; los reemplacé por mis propias mediciones y capturas.